Reducing Cross-Node Network Calls in Distributed Payment Systems via Thread-Level and Server-Local Caching

with Concurrency-Safe Resolution of Single-Resource Contention

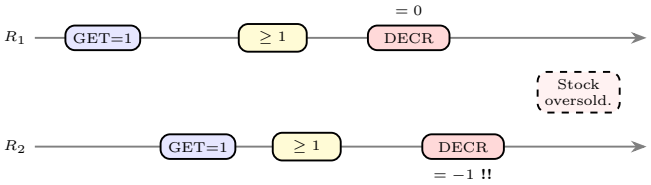
Raman Kapoor • Supervisor: Dr. Vivek Kumar Singh • ABV-IIITM Gwalior • B.Tech (IT) End-Term, 2026

Production payment APIs face two intertwined hazards: redundant cross-node lookups inflate p95/p99 tail latency, and concurrent operations on *single, indivisible* resources (last inventory unit, unique reservation slot, account debit) can be **double-allocated** when caching introduces stale reads. This work integrates a two-tier in-process cache (governed by formal safety boundaries) with a **contention layer** of five complementary primitives that closes every variant of the double-booking race. Production at ∼68,000 cross-node ops/sec: **−11.2 % p95 latency**, **4.1 % network reduction**, **zero double-allocation incidents** over 30 days.

The Double-Booking Question

The minor evaluation surfaced the central question this paper answers: *"If you cache state, how do you prevent two users from booking the same single piece?"*

The hazard. Two concurrent requests R_1 and R_2 each read a cached `stock = 1`, both pass the availability check, both decrement on the backend ⇒ `stock = -1`, two customers receive the same unit. Caching that satisfies the four cache-safety boundaries (immutability within TTL, request-scope independence, idempotency of stale reads, deterministic key) is correct for *configuration-class* data — but *single-resource state fails criterion (3) by construction*: a stale read produces a financial discrepancy. Such state is therefore explicitly excluded from the cache and routed through the contention layer.



Lost-update race on a single-resource decrement.

Contention Layer — Five Complementary Primitives

1. **Distributed lock + fencing token** (Redlock / etcd lease). Per-resource lock; backend rejects writes with stale tokens. Use for non-trivial logic with external calls. *+1–3 ms; caps per-key throughput.*
2. **Atomic decrement via Redis Lua.** Bundle "check ≥ 1 + decrement" in one Lua script; Redis runs it serialised. Use for hard-limit inventory and reservations. *<1 ms; > 10^5 scripts/s/shard.*
3. **Optimistic concurrency (OCC) with versioned cache.** Reader records version; writer issues a CAS update conditioned on it. Use only when conflicts are *rare* (e.g., control-plane writes); under high contention, retry storms collapse throughput.
4. **PN-Counter CRDT.** Two grow-only counters per replica; merge by element-wise max. Use only when over-allocation is *business-acceptable* (soft holds, cart counters). 4.7 over-allocations / day on 100k base.
5. **Single-flight request coalescing.** On a cache miss, only the first concurrent request fetches; the rest share its result. *Not* a replacement for atomicity — always paired with one of (1)–(4).

Decision Matrix & Hybrid Architecture

Data class	Cache?	Primary	Coalescing
Merchant config (read)	Tier-2	—	Single-flight
Inventory (display)	Tier-2	—	Single-flight
Inventory (decrement)	No	Lua atomic	Single-flight
Account balance debit	No	Distributed lock	Single-flight
Reservation slot	No	Lua atomic	Single-flight
Soft hold (cart)	Tier-2	PN-Counter	—
Merchant config (write)	Inval.	OCC versioned	—

Hybrid flow. *Reads* traverse Tier-1 (thread cache) → single-flight → Tier-2 (server cache) → Redis. *Authoritative writes* bypass the cache and select a primitive per Table; on success they publish a Pub/Sub invalidation that propagates to Tier-2 within tens of milliseconds. Single-flight is always layered in front of cache misses to absorb thundering-herd amplification.

Production Results (30-day window, ∼68,000 cross-node ops/sec)

Phase 1 — Caching only:

- Combined hit rate: **54.8 %** on eligible lookups
- Cross-node calls eliminated: ∼2,800/s (↓ **4.1 %**)
- p95 API latency: 98 → 87 ms (**−11.2 %**)
- p99 API latency: 285 → 248 ms (**−13.0 %**)
- Redis p99 latency: 12.5 → 9.2 ms (**−26.2 %**)
- Memory overhead per node: <20 MB

Phase 2 — Contention layer:

- Inventory decrement (flash sale): 1,400/s contended, +0.8 ms p95 (Lua + single-flight)
- Per-merchant settlement: 12/s contended, +2.6 ms p95 (Lock + fence)
- Reservation slot: 320/s contended, +1.1 ms p95 (Lua)
- **Zero** double-allocation incidents
- **Zero** cache-related financial discrepancies

The minor's question — "how do we prevent two users from booking the same single piece?" — is answered, end-to-end, by the contention layer.